Model-agnostic variable importance for survival outcomes: a simulation study under additive and mixed hazards

Simulation protocol

Introduction

We conducted a simulation study to examine the behavior of Wolock's model-agnostic exclusion variable importance measure (VIM) under a range of survival data-generating mechanisms. The objectives were to evaluate its performance and inferential validity under additive hazards, mixed additive–multiplicative hazards, and proportional hazards settings, including high-dimensional scenarios with rare binary covariates and heterogeneous effect sizes. We also compared alternative strategies for nuisance function estimation and contrasted exclusion-based importance with permutation-based importance across all scenarios.

Objectives

The simulation study was designed to evaluate the performance, robustness, and inferential validity of Wolock's model-agnostic exclusion variable importance measure (VIM) across several survival data-generating mechanisms.

First, under an **additive hazards framework**, we assess the accuracy and inferential properties of the VIM and compare different strategies for estimating the conditional survival functions, namely ***global survival stacking***, ***survival Super Learner***, the ***Aalen additive hazards model***, and ***random survival forests***.

Second, under **mixed additive and multiplicative hazard structures**, we investigate the behavior of the VIM when both types of effects coexist. In this setting, conditional survival functions are estimated using ***global survival stacking***, ***survival Super Learner***, the ***Cox–Aalen model***, and ***random survival forests***.

Third, we assess the sensitivity of the VIM when covariates correspond to **rare alterations** with **heterogeneous effect sizes**, ranging from weak to strong, under a **proportional hazards model**.

Finally, we compare **exclusion-based variable importance** with **permutation-based variable importance** to examine differences in interpretation, stability, and empirical performance across the considered scenarios.

Data-generation mechanisms

Simulation of covariates

We simulated datasets combining binary and continuous covariates for $i = 1, \dots, n$ individuals.

Binary covariates

Binary covariates were generated as correlated Bernoulli variables:

$$X_{ij} \in \{0, 1\}, \quad j = 1, \dots, p_B,$$

where $X_{ij} = 1$ indicates the presence of binary feature j.

Each variable had marginal prevalence

$$\pi_j = \Pr(X_{ij} = 1).$$

To induce correlation between variables, a latent Gaussian copula model was used:

$$\mathbf{Z}_i \sim \mathcal{N}(0, \Sigma_B),$$

where Σ_B is a block-structured correlation matrix.

Binary variables were obtained through probit thresholding:

$$X_{ij} = \mathbb{I}\left(Z_{ij} < \Phi^{-1}(\pi_j)\right),$$

where Φ^{-1} denotes the quantile function of the standard normal distribution. This construction guarantees

$$\Pr(X_{ij} = 1) = \pi_j.$$

Thus, marginal prevalences are exactly controlled while inducing dependence between binary variables through the latent correlation matrix.

Continuous covariates

Continuous covariates were simulated from a multivariate normal distribution:

$$\mathbf{W}_i \sim \mathcal{N}(0, \Sigma_C),$$

where $\Sigma_C \in \mathbb{R}^{p_C \times p_C}$ is a block-correlation matrix.

Variables were partitioned into K blocks of equal size b. For block k, define

$$B_k = \{(k-1)b + 1, \dots, kb\}.$$

The correlation matrix $\Sigma_C = (\sigma_{jl})$ was defined as

$$\sigma_{jl} = \begin{cases} 1 & \text{if } j = l, \\ \rho_k & \text{if } j \neq l \text{ and } j, l \in B_k, \\ \rho_B & \text{if } j \in B_k,\ l \in B_m,\ k \neq m. \end{cases}$$

Equivalently, each diagonal block can be written as

$$\Sigma_{C,k} = (1 - \rho_k) I_b + \rho_k \mathbf{1}_b \mathbf{1}_b^\top,$$

where I_b denotes the $b \times b$ identity matrix and $\mathbf{1}_b$ is a vector of ones of length b.

Continuous covariates were therefore correlated according to a block dependence structure.

Simulation of time-to-event outcomes

Let T denote the event time and C an independent censoring time. The observed data consisted of (Y, Δ, X, Z), where:

$$Y = min(T, C), \quad \Delta = \mathbb{I}(T \leq C).$$

Censoring times were generated independently from an exponential distribution to achieve a target censoring proportion.

Additive Aalen model

Time-to-event outcomes were generated under an additive hazards framework.
For each individual $i = 1, \ldots, n$, with covariate vector $\mathbf{X}_i$, the hazard function was defined as

$$\lambda(t \mid X_i) = \lambda_0(t) + X_i^\top \alpha,$$

where $\lambda_0(t)$ denotes the baseline hazard function and α represents the vector of additive hazard effects.

Cox-Aalen model

To evaluate performance under mixed hazard structures, we additionally generated data from a Cox-Aalen model:

$$\lambda(t|X_i, Z_i) = \lambda_0 \exp(Z_i^\top \beta) + X_i^\top \alpha.$$

where:

- Z_i are covariates with multiplicative effects,
- β are multiplicative coefficients,
- X_i are covariates with additive effects,
- α are additive coefficients

Target estimands and oracle variable importance

We considered two estimators of variable importance, targeting distinct estimands: exclusion importance and permutation importance.

Model-agnostic framework for exclusion-based variable importance in survival analysis

Definition

We consider the model-agnostic variable importance measure proposed by Wolock *et al.*, defined through covariate exclusion. For a given prediction time t_0 and covariate X_j, the VIM quantifies the change in predictive performance when X_j is removed from the prediction function.

We defined a nonparametric variable importance measure (VIM) based on the predictive value of a pathway set within a flexible survival prediction model fitted with `survML`. Let $V(.)$ denote a time-dependent predictive performance functional, evaluated on predictions. The variable importance measure for covariate X_j at time t_0, $\psi_j(t_0)$, was defined as the difference in predictive performance between a model fit using the full feature set, V_{full} and a model fit after excluding the pathway or group of pathways of interest, $V_{reduced}$:

$$\psi_j(P_0) = V_{full} - V_{reduced}$$

Estimation of the conditional survival functions

We estimated the conditional survival functions F_0 and G_0 using different strategies, depending on the scenario:

- Aalen model (`timereg::aalen()`, `pec::predictSurvProb()`)
- Cox-Aalen survival model (`timereg::cox.aalen()`, `pec::predictSurvProb()`)
- Random survival forests (`randomForestSRC::rfsrc()`)
- Survival Super Learner (`survivalSL`)
- Global survival stacking (`survML::stackG()`)

Estimation of the oracle prediction functions

We estimated the full and residual oracle prediction functions for time-horizon VIMs, f_0 and $f_{0,s}$, using Super Learner regression.

We used the sample splitting procedure to compute VIM point and standard error estimates, from which we computed nominal 95% CI Wald-type confidence intervals.

Permutation variable importance in machine learning survival models

Definition

Permutation VIM measures was calculated using `survex` package using the `model_parts()` function. The importance of j-th variable is defined as the change in the loss function $\mathcal{L}$ caused by permutation of this variable in the dataset:

$$\text{PFI}_t(f, \mathbf{X}, j, \mathcal{L}, y) = \frac{1}{B} \sum_{i=1}^{B} \Big(\mathcal{L}(f, \mathbf{X}, y) - \mathcal{L}(f, \mathbf{X}_i^{*j}, y) \Big)$$

where $\mathcal{L}$ represents the loss function chosen to evaluate model performance, $\mathbf{X}_i^{*j}$ denotes the i-th permutation of variable j within the dataset X, and B is the number of different permutations. Permuting a variable is supposed to simulate the loss of information associated with the variable. For `survex`, we used the `model_parts()` function with 10 permutations.

Model framework

We used the following models to calculates variable importance in the loss function after the variable values permutations:

- Aalen model (`timereg`, `pec`)
- Cox-Aalen survival model (`timereg`, `pec`)
- Random survival forests (`randomForestSRC`)

Time horizon

Variable importance was evaluated at a fixed time horizon, corresponding to **75th percentiles** of the empirical observed event times (or the Kaplan-Meier estimator to account for censoring). These represent a late-risk horizon Restricting the analysis to this quantile-based time point reduces computational burden while ensuring that evaluation occurs in regions with a sufficient number of individuals at risk, thereby avoiding instability in the tail of the distribution.

Performance metrics for variable importance in survival analysis

Variable importance was evaluated using the *Brier score*, $BS(t)$ to captures both discrimination and calibration. We additionally used the *cumulative/dynamic area under the ROC curve, $AUC(t)$* to estimate variable importance measures, which evaluate discrimination independently of calibration.

The ***cumulative/dynamique AUC*** at time t is defined as the probabilitu that, among a randomly selected pair consisting of one individual who experiences the event before t and one who survives beyon t, the predicted risk is higher for the former.

The ***Brier score*** at time t is the expected squared difference between the observed survival status at time t and the predicted survival probability, with inverse probability of censoring weights to account for right censoring.

Cross-validation procedure

To ensure fair comparison across nuisance estimation strategies, all models were evaluated using the same cross-validation framework.

Five-fold cross-validation was employed for out-of-sample performance assessment and to prevent optimistic bias in risk prediction and downstream VIM estimation.

We implemented K-fold cross-validation with $K = 5$. The data were randomly partitioned into five approximately equal folds. For each fold:

1. The model was trained on the $K - 1$ remaining folds.
2. Predictions of the conditional survival and hazard functions were obtained for individuals in the held-out fold.
3. Performance metrics were computed on the validation fold.

This procedure was repeated so that each observation contributed once to validation risk estimation.

Oracle variable importance

We generated a large Monte Carlo sample ($n = 200000$) from the true data-generating model to approximate population-level quantities. Using this sample, we computed the oracle predictive performance as well as the oracle VIM measure for each pathway or group of pathways.

Performance evaluation

The following metrics were computed across $B = 1000$ Monte Carlo replications.

Metric	Definition	Formula
Bias	Average deviation between the estimator and the true parameter	$\frac{1}{B}\sum_{b=1}^{B}(\hat{\theta}^{(b)} - \theta)$
RMSE	Root mean squared error measuring overall estimation accuracy	$\sqrt{\frac{1}{B}\sum_{b=1}^{B}(\hat{\theta}^{(b)} - \theta)^2}$
Coverage	Proportion of confidence intervals containing the true parameter	$\frac{1}{B}\sum_{b=1}^{B}\mathbf{1}(\theta \in CI^{(b)})$
CI width	Average length of confidence intervals	$\frac{1}{B}\sum_{b=1}^{B}(U^{(b)} - L^{(b)})$
Type I Error	Probability of rejecting H_0 when it is true	$\frac{1}{B}\sum_{b=1}^{B}\mathbf{1}(p^{(b)} < \alpha)$ under H_0

Metric	Definition	Formula
Power	Probability of rejecting H_0 when the alternative is true	$\frac{1}{B}\sum_{b=1}^{B}\mathbf{1}(p^{(b)}<\alpha)$ under H_1
Rank correlation	Correlation between true and estimated parameter rankings	
Runtime	Average computational time per replication	$\frac{1}{B}\sum_{b=1}^{B}t^{(b)}$

where:

- B is the number of Monte Carlo replications,
- θ is the true parameter,
- $\hat{\theta}^{(b)}$ is the estimate in replication b,
- $CI^{(b)} = [L^{(b)}, U^{(b)}]$,
- α is the nominal significance level.

Simulation scenarios

The main scenarios are summarized in Table 2.

Table 2: Summary of the main scenarios

Scenario n°	Effect type	Simulation of covariates	Coefficient	Nuisance function estimation
1	Additive hazards (Aalen model)	$X_1 \sim Bernoulli(0.4)$ X_2 X_3 X_4	$\alpha_1 = 0.1$ $\alpha_2 = 0.05$ $\alpha_3 = 0.005$ $\alpha_4 = 0$	Aalen model Survival Super Learner Global survival stacking
2	Mixed additive and multiplicative hazards (Cox-Aalen model)	X_1 X_2 X_3 Z_1 Z_2 Z_3	$\alpha_1 = 0.1$ $\alpha_2 = 0.05$ $\alpha_3 = 0$ $\beta_1 = 1$ $\beta_2 = 0.5$ $\beta_3 = 0$	Cox-Aalen model Survival Super Learner Global survival stacking

Scenario 1 - Additive hazards (baseline benchmark)

Objective

This baseline scenario evaluates the robustness of Wolock's model-agnostic variable importance measure (VIM) under correct model specification. Specifically, we assess whether competing nuisance-function estimators recover the true exclusion VIM when the data-generating mechanism follows an **additive hazards model**.

Data generating mechanism

Four binary covariates were generated *independently* with prevalences spanning rare to frequent alterations, $X_j \sim Bernoulli(0.4)$.

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0(t) + \alpha_1(t)X_1 + \alpha_2(t)X_2 + \alpha_3(t)X_3 + \alpha_4(t)X_4$$

with $\lambda_0(t) = 0.05$ (constant) and $\alpha_j(t) = (0.1, 0.05, 0.005, 0)$.

Independent censoring times were generated as $C \sim Uniform(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

Nuisance functions estimators to compare

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Aalen additive hazards model (`timereg::aalen`)
- Survival Super Learner (`survivalSL`)
- Global survival stacking (`survML`)

Since the data-generating mechanism follows an additive hazards model, the additive Aalen estimator is correctly specified. The remaining methods introduce varying degrees of model misspecification, allowing us to assess the robustness of the VIM procedure to nuisance-model choice.

Scenario 2 - Mixed additive and multiplicative effects

Objective

This scenario evaluate the sensitivity of the exclusion VIM when the true hazard follows a **Cox-Aalen model**.

Data-generating mechanism

The covariates are generated *independently* as follows:

- $X = (X_1, X_2, X_3, Z_1, Z_2, Z_3)$

- $X_1, X_2, X_3 \sim \mathcal{B}(0.4)$
- $Z_1, Z_2 \sim \mathcal{B}(0.4)$
- $Z_3 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1, X_2 and X_3 with coefficients $\alpha_1(t) = 0.1$, $\alpha_2 = 0.05$ and $\alpha_3(t) = 0$, respectively. Multiplicative (proportional hazards) effects were assigned to X_1, X_2 and X_3 with coefficients $\beta_1 = 1$, $\beta_2 = 0.5$ and $\beta_3 = 0$.

Independent censoring times were generated as $C \sim Uniform(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

Nuisance functions estimators to compare

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Cox-Aalen model (`timereg::cox.aalen`)
- Survival Super Learner (`survivalSL`)
- Global survival stacking (`survML`)

Scenario 3 - Rare alterations with

Objective

This scenario aims to evaluate the overall performance of the VIM procedure to ***rare alterations*** exhibiting ***heterogeneous effect sizes (ranging from weak to strong)*** under a ***proportional hazards model***. More specifically, the objective is to assess whether the VIM approach can:

- detect rare yet highly prognostic alterations,
- discriminate between strong and moderate prognostic signals, and
- correctly identify non-informative (null) variables.

Data-generating mechanisms

All covariates were generated independently as binary variables:

$$X_j \sim \mathcal{B}(\pi_j)$$

with $\pi \in \{0.01, 0.1\}$ to reflect genomic rarity.

The event times follows a Aalen or Cox-Aalen model (cf. scenario 1 and 2).

Covariates were partitioned according to both their prevalence and prognostic effect size, resulting in the following combinations:

Prevalence category	π_j	Effect category	α	β
Very rare	$\pi = 0.01$	Strong	0.10	1.0
Very rare	$\pi = 0.01$	Moderate	0.05	0.5
Very rare	$\pi = 0.01$	Null	0.00	0.0
Rare	$\pi = 0.10$	Strong	0.10	1.0
Rare	$\pi = 0.10$	Moderate	0.05	0.5
Rare	$\pi = 0.10$	Null	0.00	0.0

This design allows us to assess whether the VIM procedure can detect rare but strongly prognostic alterations, distinguish strong from moderate effects, and correctly assign low importance to null variables.

Independent censoring times were generated as $C \sim Uniform(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

Sensitivity to covariate correlations

For each scenario, we additionally evaluated the sensitivity of the VIM to correlations between covariates. Correlations were introduced between selected covariates while preserving their marginal distributions (binary or continuous). Specifically, we considered both weak and moderate correlation dependence between predictors, with pairwise correlations set to $\rho = 0.3$ and $\rho = 0.6$, respectively. This analysis allows us to assess the robustness of the estimated VIM in the presence of correlated covariates.

Compare exclusion variable importance to permutation variable importance

Objective

To examine differences in interpretation, stability, and empirical performance across the considered scenarios.

Scenarios

- Additive hazards
- Mixed additive and multiplicative hazards

True values of variable importance for all scenarios

The approximate true values of variable importance based on Brier score, and AUC(t) under all scenarios considered here are provided in Table X (to be completed).

R documentation

Additive survival models

Flexible Regression Models for Survival Data: https://cran.r-project.org/web/packages/timereg/refman/timereg.html

Super Learner in survival analysis

The super learner requires 3 components:

1. A library of estimators
2. A loss function
3. A model for combining the estimators in the library

These three components should be tailored to the specific problem. The library should be built such that each estimator in the library could individually estimate the parameter of interest. The loss function should be chosen to reflect the goals of the problem. If one is interested in estimating the hazard, the loss function should ideally involve the hazard; if interest in the survival function, the loss function should involve the survival function. This may seem common sense, but we often see researchers use a hazard loss function when the interest is on the survival probability at a specific time point. The loss function should be chosen with respect to the problem you are trying to solve. The model for combining the estimators in the library is often chosen to maintain a bounded loss function for the super learner.

Super Learner for survival data by minimizing

cross-validated negative partial log-likelihood

https://github.com/kgolmakani/SuperLearner-Survival

Golmakani's approach builds a Super Learner for survival data by minimizing **cross-validated negative partial log-likelihood**. So, the base learners are hazard-based learners optimized on Cox partial likelihood. This approach optimizes the best convex combination for **hazard ranking**.

Survival Super Learner

Predicting survival by a Super Learner (`survivalSL`)

Name	Description
LIB_COXall	Proportional hazards (PH) model with all covariates (#)
LIB_COXaic	PH model with covariate selection by AIC minimization (#)
LIB_COXen	PH model with B-spline for quantitative covariates and Elastic-Net penalization (#; hyperparameters: alpha, lambda)
LIB_COXlasso	PH model with B-spline for quantitative covariates and Lasso penalization (#; hyperparameter: lambda)
LIB_COXridge	PH model with B-spline for quantitative covariates and Ridge penalization (#; hyperparameter: lambda)
LIB_AFTgamma	Accelerated failure time (AFT) model with Gamma distribution
LIB_AFTggamma	AFT model with generalized Gamma distribution
LIB_AFTllogis	AFT model with log-logistic distribution
LIB_AFTweibull	AFT model with Weibull distribution
LIB_PHexponential	Parametric PH model with Exponential distribution
LIB_PHgompertz	Parametric PH model with Gompertz distribution
LIB_PHspline	PH model with natural cubic spline baseline (hyperparameter: k)
LIB_RSF	Random survival forest (hyperparameters: nodesize, mtry, ntree)
LIB_PLANN	One-layer survival neural network (hyperparameters: n.nodes, decay, batch.size, epochs)

Super Learning for conditional survival functions with right-censored data (`survSuperLearner`)

```
 [1] "survSL.coxph"     "survSL.expreg"    "survSL.gam"       "survSL.km"
 [5] "survSL.loglogreg" "survSL.pchreg"    "survSL.pchSL"     "survSL.rfsrc"
 [9] "survSL.template"  "survSL.weibreg"
```

```
[1] "All"
[1] "survscreen.glmnet"   "survscreen.marg"     "survscreen.template"
```

Wrapper	Description
survSL.coxph	Cox proportional hazards model
survSL.expreg	Parametric exponential survival regression
survSL.gam	Generalized additive survival model
survSL.km	Kaplan–Meier estimator
survSL.loglogreg	Log-logistic survival regression
survSL.pchreg	Piecewise constant hazards regression
survSL.pchSL	SuperLearner with piecewise constant hazards
survSL.rfsrc	Random survival forest (randomForestSRC)
survSL.template	Template for custom survival wrapper
survSL.weibreg	Weibull survival regression

Wrapper	Description
All	No screening (all variables retained)
survscreen.glmnet	Screening via glmnet penalized regression
survscreen.marg	Marginal screening (univariate survival association)
survscreen.template	Template for custom survival screening

Global survival stacking

```
 [1] "SL.bartMachine"      "SL.bayesglm"         "SL.biglasso"
 [4] "SL.caret"            "SL.caret.rpart"      "SL.cforest"
 [7] "SL.earth"            "SL.gam"              "SL.gbm"
[10] "SL.glm"              "SL.glm.interaction"  "SL.glmnet"
[13] "SL.ipredbagg"        "SL.kernelKnn"        "SL.knn"
```

```
[16] "SL.ksvm"             "SL.lda"              "SL.leekasso"
[19] "SL.lm"               "SL.loess"            "SL.logreg"
[22] "SL.mean"             "SL.nnet"             "SL.nnls"
[25] "SL.polymars"         "SL.qda"              "SL.randomForest"
[28] "SL.ranger"           "SL.ridge"            "SL.rpart"
[31] "SL.rpartPrune"       "SL.speedglm"         "SL.speedlm"
[34] "SL.step"             "SL.step.forward"     "SL.step.interaction"
[37] "SL.stepAIC"          "SL.svm"              "SL.template"
[40] "SL.xgboost"
```

```
[1] "All"
[1] "screen.corP"           "screen.corRank"        "screen.glmnet"
[4] "screen.randomForest"   "screen.SIS"           "screen.template"
[7] "screen.ttest"          "write.screen.template"
```

Wrapper	Type
SL.bartMachine	Bayesian Additive Regression Trees (BART)
SL.bayesglm	Bayesian generalized linear model
SL.biglasso	Lasso / Elastic-Net (biglasso)
SL.caret	caret unified interface
SL.caret.rpart	caret interface for rpart
SL.cforest	Conditional inference forest
SL.earth	Multivariate Adaptive Regression Splines (MARS)
SL.gam	Generalized additive model
SL.gbm	Gradient boosting machine
SL.glm	Generalized linear model

Wrapper	Type
SL.glm.interaction	GLM with interaction terms
SL.glmnet	Elastic-Net regularized GLM
SL.ipredbagg	Bagged trees (ipred)
SL.kernelKnn	Kernel k-nearest neighbors
SL.knn	k-nearest neighbors
SL.ksvm	Kernel support vector machine
SL.lda	Linear discriminant analysis
SL.leekasso	Leekasso variable selection
SL.lm	Linear model
SL.loess	Local regression (LOESS)
SL.logreg	Logistic regression
SL.mean	Mean predictor
SL.nnet	Neural network
SL.nnls	Non-negative least squares
SL.polymars	Polynomial MARS
SL.qda	Quadratic discriminant analysis
SL.randomForest	Random forest
SL.ranger	Fast random forest (ranger)
SL.ridge	Ridge regression
SL.rpart	CART decision tree
SL.rpartPrune	Pruned CART tree
SL.speedglm	Fast GLM
SL.speedlm	Fast linear model
SL.step	Stepwise selection
SL.step.forward	Forward stepwise selection

Wrapper	Type
SL.step.interaction	Stepwise with interactions
SL.stepAIC	Stepwise AIC selection
SL.svm	Support vector machine
SL.template	Template for custom wrapper
SL.xgboost	Extreme gradient boosting

Wrapper	Type
All	No screening (all variables retained)
screen.corP	Correlation screening (p-value based)
screen.corRank	Correlation rank screening
screen.glmnet	Screening via glmnet
screen.randomForest	Screening via random forest importance
screen.SIS	Sure Independence Screening
screen.template	Template for custom screening
write.screen.template	Utility to create screening template

Permutation variable importance

https://cran.r-project.org/web/packages/survex/refman/survex.html

Random survival forests

Random survival forests

Ensemble methods for survival function estimation with time-varying covariates (`LTRCforests`)

References